

Supervision and Registry — A Readable Companion to the Elixir Docs

Chapter 5 · checked against Elixir 1.20 · examples run on Elixir 1.14 and 1.18 · Sep 26, 2026

1. Why supervision is structure, not error handling

A supervisor starts processes, watches them, and restarts them from a clean state when they fail. The principles are the same as in Erlang (Erlang book, chapter 1): arrange processes so that dependencies are explicit, failures are contained, and the parts that must not fail are small.

What Elixir adds is convenience: child specs generated by `use`, the `{Module, arg}` shorthand, `DynamicSupervisor` for children started on demand, and `Registry` for finding processes by name. Convenience means defaults, and several of those defaults decide what happens in exactly the situation supervision exists for.

2. The building blocks

Child specs

`use GenServer`, `use Agent`, `use Task` and `use Supervisor` all generate `child_spec/1`, so a child list is usually just `{Module, arg}` tuples:

```
children = [
  {Registry, keys: :unique, name: MyApp.Registry},
  {DynamicSupervisor, name: MyApp.SessionSup, strategy: :one_for_one},
  {MyApp.Cache, ttl: 60}
]
Supervisor.start_link(children, strategy: :rest_for_one)
```

Key	Default	Meaning
<code>:id</code>	The module name	Must be unique within one supervisor
<code>:restart</code>	<code>:permanent</code>	<code>:permanent</code> always, <code>:transient</code> if abnormal exit, <code>:temporary</code> never
<code>:shutdown</code>	5000 for workers, <code>:infinity</code> for supervisors	How long to wait before killing the child
<code>:type</code>	<code>:worker</code>	<code>:supervisor</code> for nested supervisors

Override per module with `use GenServer, restart: :temporary`, or per use with `Supervisor.child_spec({Mod, arg}, id: :other, restart: :transient)`.

Supervisors

Kind	Children	Restart limit default	Use for
<code>Supervisor</code>	Fixed list, started in order	3 restarts in 5 s	The static structure of your application
<code>DynamicSupervisor</code>	Started on demand with <code>start_child/2</code>	3 restarts in 5 s, for <i>all</i> children together	Sessions, connections, jobs
<code>PartitionSupervisor</code> (1.14+)	N copies of a child, keyed by partition	Per partition	Spreading load or failure across several supervisors
<code>Task.Supervisor</code>	Tasks	3 restarts in 5 s	Supervised one-off work (chapter 3)

Strategies are the Erlang ones: `:one_for_one`, `:rest_for_one`, `:one_for_all`. Children start in list order and shut down in reverse.

Registry

A Registry is a local, ETS-backed key-process store. With keys: `:unique` it names processes:

```
name = {:via, Registry, {MyApp.Registry, {:session, user_id}}}
DynamicSupervisor.start_child(MyApp.SessionSup, {Session, name: name})
GenServer.call(name, :get)
```

With keys: `:duplicate` it groups processes under a key, which is the basis for local pub-sub with `Registry.dispatch/3`.

3. How it actually works

Restart limits are per supervisor

`max_restarts: 3`, `max_seconds: 5` counts restarts across *all* the supervisor's children. When the limit is exceeded the supervisor terminates every child and then itself (Erlang book, chapter 1). For a `Supervisor` with five distinct services this is sensible. For a `DynamicSupervisor` with thousands of independent children, it means a handful of unrelated crashes takes down every one of them.

Registry cleanup is eventually consistent

When a registered process dies, the Registry removes its entries, but not atomically with the death. The docs warn that a pid can die “at any time, including between getting the value from the registry and sending it a message.” A lookup can return a dead pid, and a new process registering the same key immediately may briefly see the old entry.

Order is dependency

Children start strictly in order, and each child's `init/1` finishes before the next starts. A child that uses the Registry must come after the Registry. With `:rest_for_one`, a restart of the Registry also restarts everything after it, which is usually exactly what dependents need.

4. Plausible but wrong

4.1 A few crashes take down every session

```
children = [{DynamicSupervisor, name: SessionSup, strategy: :one_for_one}]
# ... thousands of sessions via DynamicSupervisor.start_child(SessionSup, {Session, id})
```

Sessions are `:permanent` by default, and the supervisor allows 3 restarts in 5 seconds for all of them. In a test with 1,000 children, 4 crashes from bad input killed the `DynamicSupervisor` and **all 1,000** children (verified). Fix: make per-session processes `restart: :temporary` (clients reconnect), and if some must restart, raise `max_restarts` deliberately or spread them with a `PartitionSupervisor` so failures stay contained.

4.2 Two children with the same id

```
children = [{Worker, :orders}, {Worker, :payments}]
Supervisor.start_link(children, strategy: :one_for_one)
```

Both children get `id: Worker`, and the supervisor refuses to start (verified: `{:error, ...}`). Under an application this fails the whole boot with an error about duplicate child specifications. Fix: `Supervisor.child_spec({Worker, :orders}, id: :orders_worker)` for each.

4.3 Using the Registry before it exists

```
children = [
  {DynamicSupervisor, name: SessionSup},
  {SessionPreloader, []}, # starts sessions named via Registry
  {Registry, keys: :unique, name: MyApp.Registry}
]
```

`SessionPreloader` starts before the Registry, so its via-tuple names fail and boot crashes. It works in a test that starts the Registry separately. Fix: put the Registry first and use `:rest_for_one`, so a Registry restart also restarts everything that depends on it.

4.4 Assuming a lookup finds a live process

```
def notify(user_id, msg) do
  [{pid, _}] = Registry.lookup(MyApp.Registry, {:session, user_id})
```

```
    send(pid, msg)
  end
```

If the user has no session, the match raises. If the session just died, the lookup may still return its pid, and `send/2` silently goes nowhere. Fix: handle `[]` explicitly. For requests, call through the via-tuple (`GenServer.call({:via, Registry, ...}, msg)`), which exits with `:noproc` when nobody is there, instead of a silent `send`.

4.5 Restarting into an old registration

```
def restart_session(user_id) do
  [{pid, _}] = Registry.lookup(MyApp.Registry, {:session, user_id})
  :ok = DynamicSupervisor.terminate_child(SessionSup, pid)
  DynamicSupervisor.start_child(SessionSup, {Session, name: via(user_id)})
end
```

Because Registry cleanup is eventually consistent, the new process can find the old registration still present and fail with `{:error, {:already_registered, old_pid}}`. It depends on timing, so it passes locally and fails under load. Fix: treat `already_registered` as a normal outcome (retry briefly, or reuse the existing process), or design so that the same process is kept rather than replaced.

4.6 Supervising per-request processes as permanent

```
defmodule Upload do
  use GenServer      # :permanent by default
  def init(%{socket: s} = st), do: {:ok, st}
end
```

When an upload crashes because the client vanished, the supervisor restarts it with the same arguments, including a socket that no longer exists. It crashes again straight away, consuming the restart budget (4.1). Fix: use `GenServer`, `restart: :temporary` for anything tied to an external session; use `:transient` for jobs that should retry after failure but not after finishing normally.

4.7 Cleanup that never runs at shutdown

```
def terminate(_reason, state) do
  flush_buffer(state)
end
```

`terminate/2` runs on supervisor shutdown only if the process traps exits (Erlang book, chapter 1, section 4.1). Without that, the buffer is lost on every deploy. Fix: `Process.flag(:trap_exit, true)` in `init/1`, a `:shutdown` value large enough for the flush, and remember that `terminate/2` is still not guaranteed if the process is killed.

5. Review checklist and sources

- ❑ Children are ordered by dependency; Registries come before their users; `:rest_for_one` where later children depend on earlier ones
- ❑ Every child in a supervisor has a unique `:id`
- ❑ Per-connection and per-request processes are `restart: :temporary`
- ❑ `DynamicSupervisor` restart limits are set deliberately for the number of children, or children are spread with `PartitionSupervisor`
- ❑ Registry lookups handle `[]` and dead pids; requests go through via-tuples
- ❑ Re-registering a key handles `{:error, {:already_registered, _}}`
- ❑ Processes with cleanup trap exits and have a suitable `:shutdown`

Sources

- Supervisor: <https://hexdocs.pm/elixir/Supervisor.html>
- `DynamicSupervisor`: <https://hexdocs.pm/elixir/DynamicSupervisor.html>
- `PartitionSupervisor`: <https://hexdocs.pm/elixir/PartitionSupervisor.html>
- Registry: <https://hexdocs.pm/elixir/Registry.html>
- Erlang book, chapter 1 (Supervision Trees): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>